

### 3 线程的优先级及线程的调度

# 线程优先级的概念

同一时刻如果有多个线程处于可运行状态，则它们需要排队等待 CPU 资源。此时每个线程自动获得一个线程的优先级，优先级的高低反映线程的重要或紧急程度。

1

可运行状态的线程按优先级排队，线程调度采用优先级调度策略，优先级高的线程应该有更大的获取 CPU 资源执行的概率。

2

当前正在执行的线程，其优先级一般不会比处于等待状态的线程优先级低。

3

# 线程优先级的设置

Java 中线程的优先级用 1-10 之间的整数表示，数值越大优先级越高，默认优先级为 5。

当需要改变线程的优先级时，可以通过调用 `setPriority()` 方法来实现，下面为该方法的声明：

```
public final void  
setPriority(int newPriority)
```

参数 `newPriority` 表示需要设置的优先级别，应该是 1-10 之间的整数。

为了便于记忆，Java 中也提供了 3 个常量来表示比较常用的优先级别，分别为

`MAX_PRIORITY`, `NORM_PRIORITY`, `MIN_PRIORITY`，分别代表数值 10, 5, 1。

# 有关线程优先级的程序示例

## 线程类的子类— Thread1 类

```
class Thread1 extends Thread
{
    public void run()
    {
        for(int i=1;i<=10000;i++)
            { System.out.print("<Min"+i+">");
            }
    }
}
```

## 线程类的子类— Thread2 类

```
class Thread2 extends Thread
{
    public void run()
    {
        for(int i=1;i<=10000;i++)
            { System.out.print("<Max"+i+">");
            }
    }
}
```

## 主类 ( 测试类 )—

### testThread

```
public class testThread {  
    public static void main(String args[])  
    {  
        // 创建两个线程对象  
        Thread1 t1=new Thread1();  
        Thread2 t2=new Thread2();  
        // 设置两个线程的优先级  
        t1.setPriority(Thread.MIN_PRIORITY);  
        t2.setPriority(Thread.MAX_PRIORITY);  
        // 启动两个线程  
        t1.start();  
        t2.start();    }    }
```

## 程序运行结果及分析

程序运行结  
果：

```
C:\Windows\system32\cmd.exe
84><Min9785><Min9786><Min9787><Min9788><Min9789><Min9790><Min9791><Min9792><Min9793><Min9794><Min9795><Min9796><Min9797><Min9798><Min9799><Min9800><Min9801><Min9802><Min9803><Min9804><Min9805><Min9806><Min9807><Min9808><Min9809><Min9810><Min9811><Min9812><Min9813><Min9814><Min9815><Min9816><Min9817><Min9818><Min9819><Min9820><Min9821><Min9822><Min9823><Min9824><Min9825><Min9826><Min9827><Min9828><Min9829><Min9830><Min9831><Min9832><Min9833><Min9834><Min9835><Min9836><Min9837><Min9838><Min9839><Min9840><Min9841><Min9842><Min9843><Min9844><Min9845><Min9846><Min9847><Min9848><Min9849><Min9850><Min9851><Min9852><Min9853><Min9854><Min9855><Min9856><Min9857><Min9858><Min9859><Min9860><Min9861><Min9862><Min9863><Min9864><Min9865><Min9866><Min9867><Min9868><Min9869><Min9870><Min9871><Min9872><Min9873><Min9874><Min9875><Min9876><Min9877><Min9878><Min9879><Min9880><Min9881><Min9882><Min9883><Min9884><Min9885><Min9886><Min9887><Min9888><Min9889><Min9890><Min9891><Min9892><Min9893><Min9894><Min9895><Min9896><Min9897><Min9898><Min9899><Min9900><Min9901><Min9902><Min9903><Min9904><Min9905><Min9906><Min9907><Min9908><Min9909><Min9910><Min9911><Min9912><Min9913><Min9914><Min9915><Min9916><Min9917><Min9918><Min9919><Min9920><Min9921><Min9922><Min9923><Min9924><Min9925><Min9926><Min9927><Min9928><Min9929><Min9930><Min9931><Min9932><Min9933><Min9934><Min9935><Min9936><Min9937><Min9938><Min9939><Min9940><Min9941><Min9942><Min9943><Min9944><Min9945><Min9946><Min9947><Min9948><Min9949><Min9950><Min9951><Min9952><Min9953><Min9954><Min9955><Min9956><Min9957><Min9958><Min9959><Min9960><Min9961><Min9962><Min9963><Min9964><Min9965><Min9966><Min9967><Min9968><Min9969><Min9970><Min9971><Min9972><Min9973><Min9974><Min9975><Min9976><Min9977><Min9978><Min9979><Min9980><Min9981><Min9982><Min9983><Min9984><Min9985><Min9986><Min9987><Min9988><Min9989><Min9990><Min9991><Min9992><Min9993><Min9994><Min9995><Min9996><Min9997><Min9998><Min9999><Min10000>请按任意键继续. . .
```

**结果分析：** (1) 程序中为两个线程对象 **t1** 、 **t2** 分别设置为最低、最高优先级，在启动时，虽然先启动了优先级低的 **t1** 线程，但最终还是优先级高的 **t2** 线程首先运行完毕，然后优先级低的线程 **t1** 才运行完毕。(2) 说明了“**优先级高的线程有更大的获取 CPU 资源执行的概率**”